GESTURE LAB
FLUTTER

Nama: Ratu Alyvia Meydiandra
NIM: E41240153
Golongan: A

TEKNIK INFORMATIKA | WORKSHOP MOBILE APPLICATIONS FRAMEWORK

PENGENALAN GESTURE FLUTTER

Gesture adalah interaksi pengguna dengan layar, seperti menekan (tap), menekan dua kali (double tap), menggeser (drag), atau mencubit untuk memperbesar/memperkecil (pinch/zoom).

Di Flutter, gesture ini bisa ditangkap menggunakan widget GestureDetector, yang memungkinkan kita merespons aksi pengguna secara langsung. Selain itu, gesture juga bisa dikombinasikan dengan animasi seperti AnimatedContainer atau Transform, sehingga perubahan visual menjadi lebih halus dan interaktif.

BERIKUT JENIS JENIS GESTURE YANG AKAN DIBAHAS PADA PPT INI MELIPUTI...

- TapPage
- DragPage
- ScalePage
- PlaygroundPage

TAPPAGE | INTERAKSI TAP DI FLUTTER

- TapPage adalah halaman Flutter yang memungkinkan pengguna berinteraksi dengan kotak animasi menggunakan gesture:
- Tap: mengubah ukuran & warna kotak
- Double Tap: ubah warna menjadi hijau
- Long Press: besarkan kotak
- Menunjukkan dasar penggunaan GestureDetector dan AnimatedContainer.

STRUKTUR WIDGET UTAMA

```
Scaffold : Layout Halaman
  ├── AppBar(title: "Tap Interaction") : Judul halaman
  └── Center
      └── GestureDetector : Menangkap gesture pengguna
      └── AnimatedContainer : Kotak animasi yang berubah ukuran & warna
```

```dart
import 'package:flutter/material.dart';

class TapPage extends StatefulWidget {
  @override
  _TapPageState createState() => _TapPageState();
}

class _TapPageState extends State<TapPage> {
  double size = 150;
  Color color = Colors.blue;

  void randomize() {
    setState(() {
      size = size == 150 ? 220 : 150;
      color = color == Colors.blue ? Colors.red : Colors.blue;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Tap Interaction")),
      body: Center(
        child: GestureDetector(
          onTap: randomize,
          onDoubleTap: () => setState(() => color = Colors.green),
          onLongPress: () => setState(() => size = 250),
          child: AnimatedContainer(
            duration: Duration(milliseconds: 300),
            width: size,
            height: size,
            decoration: BoxDecoration(
              color: color,
              borderRadius: BorderRadius.circular(30),
              boxShadow: [BoxShadow(blurRadius: 20, color: Colors.black26)],
            ), // BoxDecoration
          ), // AnimatedContainer
        ), // GestureDetector
      ), // Center
    ); // Scaffold
  }
}
```

PENJELASAN KODE PROGRAM

VARIABEL

```
class _TapPageState extends State<TapPage> {
  double size = 150;
  Color color = Colors.blue;
```

Variabel size menentukan ukuran awal kotak, sedangkan color menentukan warna awalnya. Kedua variabel ini berfungsi menyimpan kondisi kotak yang bisa berubah saat gesture dijalankan.

FUNGSI TAP

```
void randomize() {
  setState(() {
    size = size == 150 ? 220 : 150;
    color = color == Colors.blue ? Colors.red : Colors.blue;
  });
}
```

Fungsi onTap adalah mengubah ukuran kotak antara 150 dan 220, sekaligus mengganti warnanya dari biru menjadi merah setiap kali ditap.

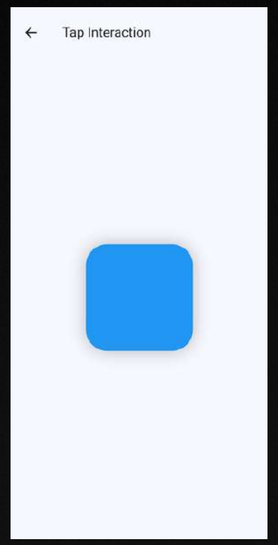
GESTUREDETECTOR

```
child: GestureDetector(
  onTap: randomize,
  onDoubleTap: () => setState(() => color = Colors.green),
  onLongPress: () => setState(() => size = 250),
  child: AnimatedContainer(
    duration: Duration(milliseconds: 300),
    width: size,
    height: size,
    decoration: BoxDecoration(
      color: color,
      borderRadius: BorderRadius.circular(30),
      boxShadow: [BoxShadow(blurRadius: 20, color: Colors.black26)],
    ), // BoxDecoration
  ), // AnimatedContainer
), // GestureDetector
```

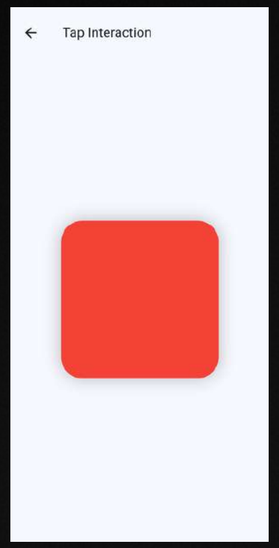
GestureDetector menangkap interaksi pengguna. Saat di-tap, kotak memanggil randomize() dan berubah ukuran serta warna. Double tap membuat kotak berwarna hijau, long press membesarkan kotak, dan semua perubahan ditampilkan dengan animasi smooth menggunakan AnimatedContainer.

HASIL PRAKTIKUM KODE PROGRAM

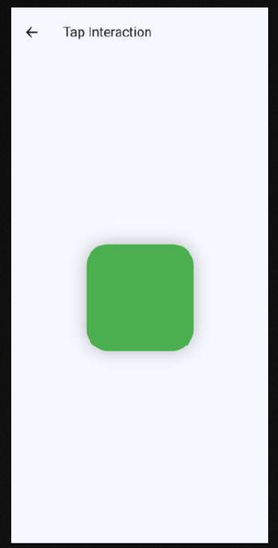
- Jalankan aplikasi Flutter di HP (melalui USB atau koneksi wireless).
- Tap kotak: kotak akan berubah ukuran dan warna.
- Double Tap: kotak berubah warna menjadi hijau.
- Long Press: kotak membesar.



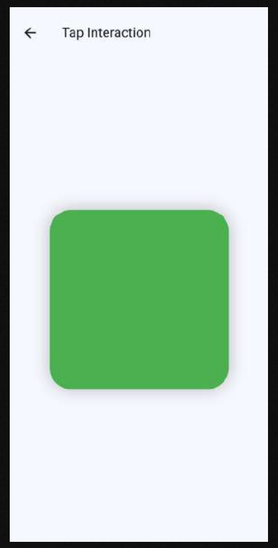
TAMPILAN AWAL



TAP KOTAK



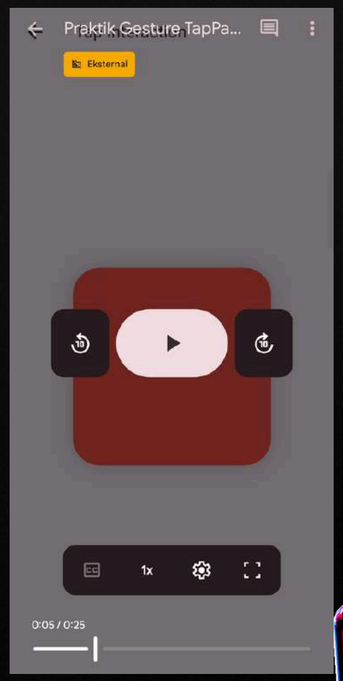
DOUBLE TAP



LONG PRESS

ADAPUN VIDEO PRAKTIKUM

Akses disini:
https://drive.google.com/file/d/1Y-a2bGX2vWofzsoGvhc2q24dVogFoppE/view?usp=drivesdk



DRAGPAGE | INTERAKSI GESER DI FLUTTER

- DragPage adalah halaman Flutter yang memungkinkan pengguna memindahkan objek dengan gesture drag.
- Drag (Pan): memindahkan posisi kotak mengikuti sentuhan jari
- Pergerakan berlangsung real-time mengikuti arah gesture
- Menggunakan dasar: GestureDetector, Stack, Positioned, AnimatedContainer.

STRUKTUR WIDGET UTAMA

Scaffold : Layout halaman
├─── AppBar(title: "Drag Movement") : Judul halaman
└─── Stack
└─── Positioned : Mengatur posisi bebas di layar
└─── GestureDetector: Membaca gerakan pengguna
└─── AnimatedContainer: Perpindahan gerakan terlihat lebih *smooth*.

```flutter/material.dart
import 'package:flutter/material.dart';

class DragPage extends StatefulWidget {
  @override
  _DragPageState createState() => _DragPageState();
}

class _DragPageState extends State<DragPage> {
  double x = 100;
  double y = 100;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Drag Movement")),
      body: Stack(
        children: [
          Positioned(
            left: x,
            top: y,
            child: GestureDetector(
              onPanUpdate: (d) {
                setState(() {
                  x += d.delta.dx;
                  y += d.delta.dy;
                });
              },
              child: AnimatedContainer(
                duration: Duration(milliseconds: 100),
                width: 120,
                height: 120,
                decoration: BoxDecoration(
                  color: Colors.orange,
                  borderRadius: BorderRadius.circular(20),
                ), // BoxDecoration
              ), // AnimatedContainer
            ), // GestureDetector
          ), // Positioned
        ],
      ), // Stack
    ); // Scaffold
  }
```

PENJELASAN KODE PROGRAM

VARIABEL POSISI

```
class _DragPageState extends State<DragPage> {
  double x = 100;
  double y = 100;
```

Digunakan untuk menyimpan posisi kotak.
x untuk horizontal (kiri-kanan), y untuk vertikal (atas-bawah). Awalnya kotak muncul di posisi (100, 100). Nilainya akan berubah saat kotak digeser.

POSITIONED

```
Positioned(
  left: x,
  top: y,
```

Bagian ini mengatur posisi kotak berdasarkan x dan y, sehingga kotak bergerak saat nilainya berubah.

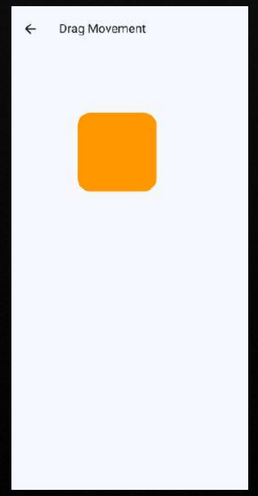
GESTUREDETECTOR DAN SETSTATE()

```
child: GestureDetector(
  onPanUpdate: (d) {
    setState(() {
      x += d.delta.dx;
      y += d.delta.dy;
    });
  },
```

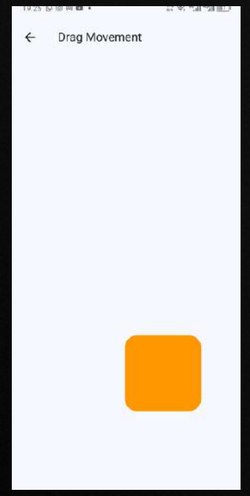
GestureDetector digunakan untuk mendeteksi sentuhan pengguna, khususnya melalui onPanUpdate yang aktif saat terjadi gerakan geser.
Parameter (d) menyimpan data pergerakan jari, yaitu d.delta.dx untuk arah horizontal dan d.delta.dy untuk arah vertikal.
Nilai tersebut kemudian ditambahkan ke variabel x dan y, sehingga posisi kotak ikut berubah sesuai arah gerakan.

HASIL PRAKTIKUM KODE PROGRAM

- Jalankan aplikasi Flutter di HP (USB / wireless)
- Drag kotak: Kotak akan bergerak mengikuti arah jari pengguna secara real-time
- Pergerakan bebas: Bisa digeser ke segala arah (atas, bawah, kiri, kanan)
- Animasi: Perpindahan posisi terlihat halus karena AnimatedContainer



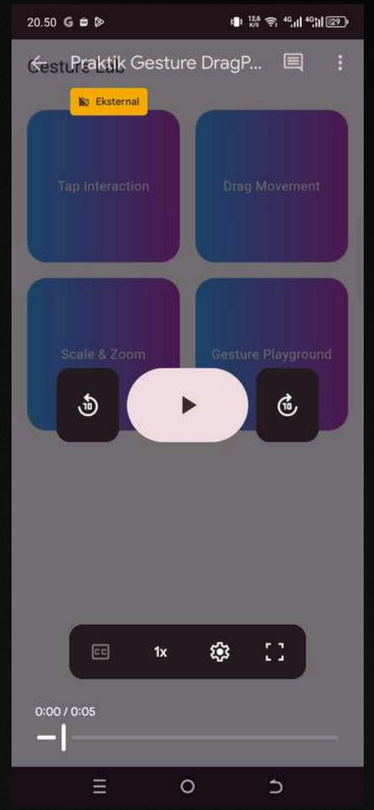
TAMPILAN AWAL



KOTAK BERGESER

ADAPUN VIDEO PRAKTIKUM

Akses disini:
https://drive.google.com/file/d/1wzvCFcQjloYvu7jml2R8-Ju1s024mxMu/view?usp=drivesdk



SCALEPAGE | INTERAKSI SCALE & ZOOM DI FLUTTER

- ScalePage adalah halaman Flutter yang memungkinkan pengguna berinteraksi dengan objek menggunakan gesture lanjutan:
- Pinch (dua jari): memperbesar dan memperkecil objek (zoom)
- Drag: menggeser posisi objek di layar
- Scroll (mouse/laptop): melakukan zoom tanpa sentuhan
- Menunjukkan penggunaan GestureDetector, Listener, dan Transform untuk interaksi yang lebih kompleks.

STRUKTUR WIDGET UTAMA

```
Scaffold : Layout Halaman
  ├── AppBar(title: "Scale & Zoom (PRO)") : Judul halaman
  └── Center
      └── Listener : Deteksi scroll mouse
      └── GestureDetector : Menangkap pinch & drag
      └── AnimatedContainer
          └── Transform.translate : Geser posisi
          └── Transform.scale : Zoom objek
```

```java
class _ScalePageState extends State<ScalePage> {
  double scale = 1.0;
  double previousScale = 1.0;

  Offset position = Offset.zero;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Scale & Zoom (PRO)")),
      body: Center(
        child: Listener(
          onPointerSignal: (pointerSignal) {
            // ✅ Zoom pakai scroll (laptop / mouse)
            if (pointerSignal is PointerScrollEvent) {
              setState(() {
                scale += pointerSignal.scrollDelta.dy * -0.001;
                scale = scale.clamp(0.5, 3.0);
              });
            }
          },
          child: GestureDetector(
            // ✅ Start pinch
            onScaleStart: (details) {
              previousScale = scale;
            },

            // ✅ Update pinch + drag
            onScaleUpdate: (details) {
              setState(() {
                // Zoom
                scale = (previousScale * details.scale).clamp(0.5, 3.0);

                // Drag (geser posisi)
                position += details.focalPointDelta;
              });
            },

            child: AnimatedContainer(
              duration: const Duration(milliseconds: 200),
              curve: Curves.easeOut,
              child: Transform.translate(
                offset: position,
```

PENJELASAN KODE PROGRAM

VARIABEL UTAMA

```
class _ScalePageState extends State<ScalePage> {
  double scale = 1.0;
  double previousScale = 1.0;

  Offset position = Offset.zero;
```

Terdapat tiga variabel utama: scale untuk mengatur ukuran objek, previousScale untuk menyimpan ukuran awal sebelum gesture, dan position untuk menyimpan posisi objek di layar.

SCROLL ZOOM (LISTENER)

```
child: Listener(
  onPointerSignal: (pointerSignal) {
    // ✅ Zoom pakai scroll (laptop / mouse)
    if (pointerSignal is PointerScrollEvent) {
      setState(() {
        scale += pointerSignal.scrollDelta.dy * -0.001;
        scale = scale.clamp(0.5, 3.0);
      });
    }
  },
```

Listener menangkap scroll mouse untuk melakukan zoom, sehingga objek bisa diperbesar atau diperkecil tanpa sentuhan layar.

START PINCH GESTURE

```
    // ✅ Start pinch
    onScaleStart: (details) {
      previousScale = scale;
    },
```

Saat pinch dimulai, onScaleStart menyimpan nilai awal scale.

UPDATE PINCH & DRAG

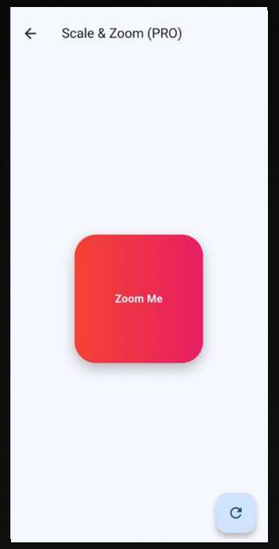
```
    // ✅ Update pinch + drag
    onScaleUpdate: (details) {
      setState(() {
        // Zoom
        scale = (previousScale * details.scale).clamp(0.5, 3.0);

        // Drag (geser posisi)
        position += details.focalPointDelta;
      });
    },
```

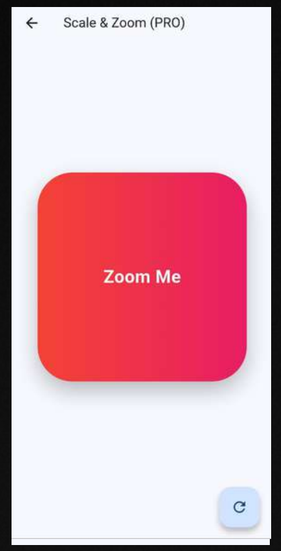
onScaleUpdate digunakan untuk memperbesar atau memperkecil objek melalui details.scale dan menggeser posisi objek dengan details.focalPointDelta. Semua perubahan langsung diperbarui menggunakan setState().

HASIL PRAKTIKUM KODE PROGRAM

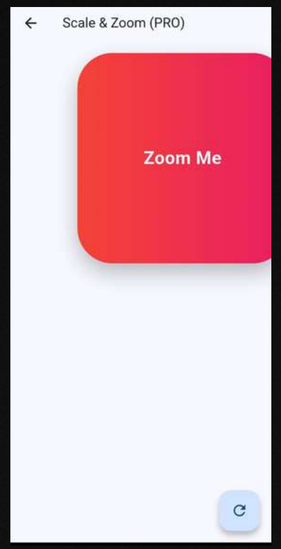
- Jalankan aplikasi Flutter di HP (melalui USB atau koneksi wireless).
- Pinch (dua jari): objek dapat di-zoom in dan zoom out
- Drag: objek dapat digeser ke berbagai arah
- Scroll (laptop/mouse): objek bisa di-zoom tanpa sentuhan
- Tombol reset: mengembalikan ukuran dan posisi ke awal



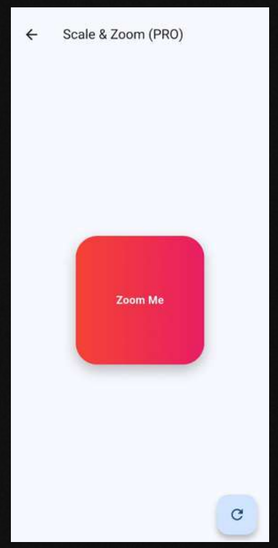
TAMPILAN AWAL



ZOOM KOTAK



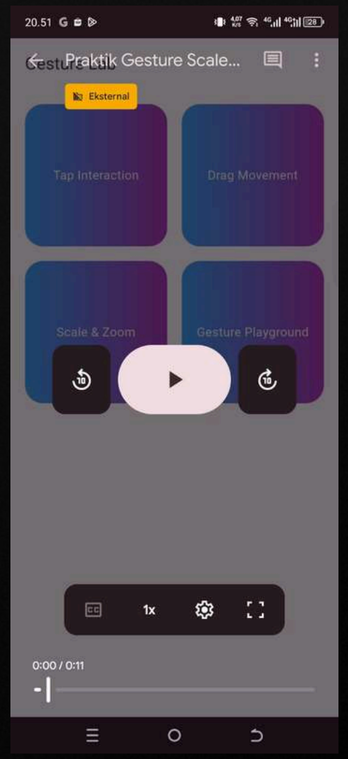
GESER KOTAK



RESET

ADAPUN VIDEO PRAKTIKUM

Akses disini: https://drive.google.com/file/d/1BIXdx_64iabK8Z3cepzH0ggxUalL2GN1/view?usp=drivesdk



PLAYGROUNDPAGE | GESTURE PLAYGROUND DI FLUTTER

- PlaygroundPage adalah halaman Flutter yang menggabungkan beberapa gesture sekaligus:
- Tap: mengubah warna kotak menjadi hijau
- Double Tap: mengubah warna kotak menjadi merah
- Long Press: membesarkan kotak
- Pinch (Scale) + Drag: memperbesar/memperkecil dan menggeser kotak
- Menunjukkan kombinasi gesture dasar dan lanjutan menggunakan GestureDetector, Transform, dan AnimatedContainer.

STRUKTUR WIDGET UTAMA

```
Scaffold : Layout Halaman
  ├── AppBar(title: "Gesture Playground") : Judul halaman
  └── Stack
      └── Positioned : Menentukan posisi awal kotak
          └── GestureDetector : Menangkap Tap, Double Tap, Long Press, Scale + Drag
              └── Transform.scale : Zoom kotak
              └── AnimatedContainer : Kotak animasi dengan warna & shadow
```

```dart
class _PlaygroundPageState extends State<PlaygroundPage> {
  double x = 100, y = 100;

  double scale = 1.0;
  double previousScale = 1.0;

  Color color = Colors.blue;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Gesture Playground")),
      body: Stack(
        children: [
          Positioned(
            left: x,
            top: y,
            child: GestureDetector(
              // TAP
              onTap: () => setState(() => color = Colors.green),

              // DOUBLE TAP
              onDoubleTap: () => setState(() => color = Colors.red),

              // LONG PRESS
              onLongPress: () => setState(() => scale = 2.0),

              // SCALE START (penting!)
              onScaleStart: (details) {
                previousScale = scale;
              },

              // SCALE + DRAG (gabungan)
              onScaleUpdate: (details) {
                setState(() {
                  // Zoom
                  scale = (previousScale * details.scale).clamp(0.5, 3.0);

                  // Drag
                  x += details.focalPointDelta.dx;
                  y += details.focalPointDelta.dy;
                });
              },
```

PENJELASAN KODE PROGRAM

VARIABEL UTAMA

```
class _PlaygroundPageState extends State<PlaygroundPage> {
  double x = 100, y = 100;

  double scale = 1.0;
  double previousScale = 1.0;

  Color color = Colors.blue;
```

x dan y menyimpan posisi kotak, scale menyimpan ukuran saat ini, previousScale menyimpan ukuran sebelum pinch, dan color menyimpan warna kotak.

TAP, DOUBLE TAP, LONG PRESS

```
child: GestureDetector(
  // TAP
  onTap: () => setState(() => color = Colors.green),

  // DOUBLE TAP
  onDoubleTap: () => setState(() => color = Colors.red),

  // LONG PRESS
  onLongPress: () => setState(() => scale = 2.0),
```

Tap mengubah warna kotak menjadi hijau, double tap menjadi merah, dan long press membesarkan kotak.

PINCH + DRAG

```
// SCALE START (penting!)
onScaleStart: (details) {
  previousScale = scale;
},

// SCALE + DRAG (gabungan)
onScaleUpdate: (details) {
  setState(() {
    // Zoom
    scale = (previousScale * details.scale).clamp(0.5, 3.0);

    // Drag
    x += details.focalPointDelta.dx;
    y += details.focalPointDelta.dy;
  });
},
```

Pinch memperbesar atau memperkecil kotak, sementara drag menggeser posisi kotak di layar. Semua perubahan langsung diterapkan dengan setState().

RESET BUTTON

```
floatingActionButton: FloatingActionButton(
  onPressed: () {
    setState(() {
      x = 100;
      y = 100;
      scale = 1.0;
      previousScale = 1.0;
      color = Colors.blue;
    });
  },
  child: Icon(Icons.refresh),
),  // FloatingActionButton
```

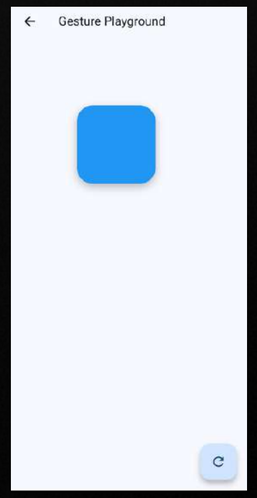
Tombol reset mengembalikan posisi, ukuran, dan warna kotak ke kondisi awal.

HASIL PRAKTIKUM KODE PROGRAM

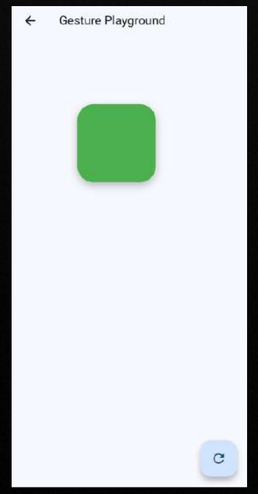
- Jalankan aplikasi Flutter di HP (USB atau wireless):
- Tap kotak: berubah hijau
- Double Tap: berubah merah
- Long Press: membesar
- Pinch: zoom in/out
- Geser jari: kotak ikut berpindah
- Reset: kembalikan ukuran, posisi, dan warna

ADAPUN VIDEO PRAKTIKUM

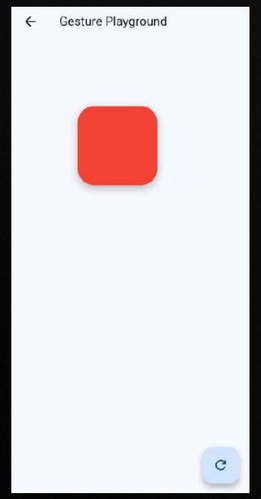
Akses disini:
https://drive.google.com/file/d/1-jCIUNeQnH_cY4xXMrU9iwxDb9KakSzf/view?usp=drivesdk



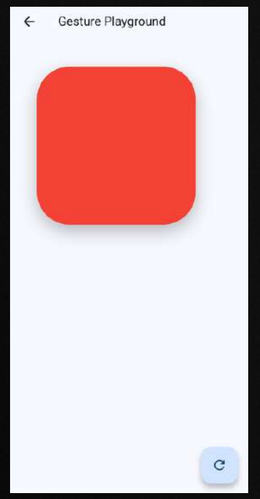
TAMPILAN AWAL



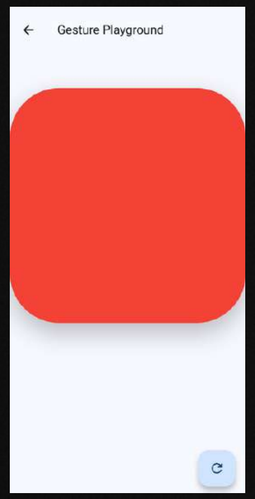
TAP KOTAK



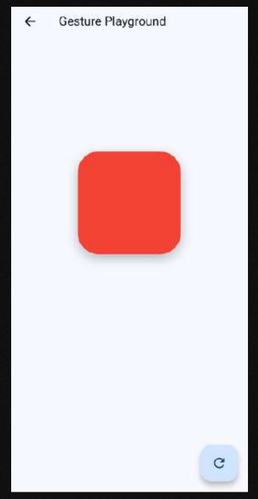
DOUBLE TAP



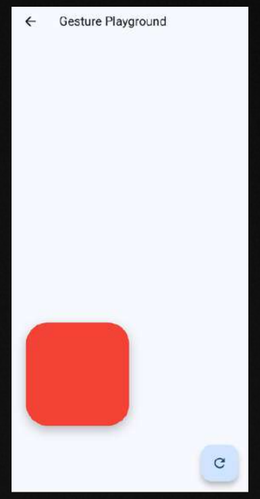
LONG PRESS



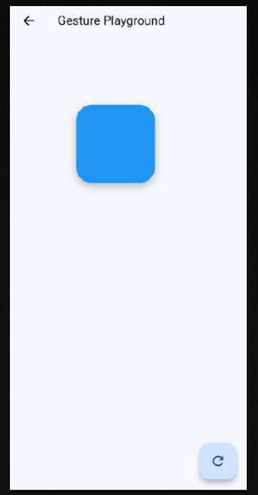
ZOOM IN



ZOOM OUT



GESER KOTAK



RESET

MAIN.DART & GESTUREAPP

Main.dart adalah titik awal aplikasi Flutter. Di sini, runApp(GestureApp()) menjalankan aplikasi, sedangkan GestureApp adalah StatelessWidget yang mengatur tema aplikasi melalui ThemeData dan menonaktifkan debug banner (debugShowCheckedModeBanner: false). Yang paling penting, GestureApp menentukan halaman utama (home: HomePage()), sehingga ketika aplikasi dijalankan, Flutter langsung menampilkan menu utama dan semua fitur gesture bisa diakses.

```dart
import 'package:flutter/material.dart';
import 'home_page.dart';

Run | Debug | Profile
void main() {
  runApp(GestureApp());
}

class GestureApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gesture Lab',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.blue),
      home: HomePage(),
    ); // MaterialApp
  }
}
```

Variabel home: HomePage() adalah kunci agar halaman menu muncul pertama kali, sedangkan ThemeData dan debugShowCheckedModeBanner memastikan tampilan konsisten dan bersih.

Intinya:
main.dart adalah pintu masuk aplikasi, yang mengatur tema dan membuka halaman utama untuk mengakses semua fitur gesture.

HOMEPAGE (MENU UTAMA APLIKASI)

HomePage menampilkan semua fitur gesture dengan kotak menu rapi dua kolom. Setiap kotak bisa ditekan (GestureDetector) untuk membuka halaman fitur (Navigator.push(...)) dan dilengkapi animasi halus (AnimatedContainer) agar tampilan lebih menarik.

Array menus menyimpan judul dan halaman masing-masing fitur, GridView.builder menata kotak menu, dan GestureDetector menghubungkan interaksi pengguna dengan navigasi ke halaman gesture.

Intinya:
HomePage menghubungkan semua fitur, membuat aplikasi interaktif, mudah dijelajahi, dan tampilannya rapi.

```
class HomePage extends StatelessWidget {
  final menus = [
    {"title": "Tap Interaction", "page": TapPage()},
    {"title": "Drag Movement", "page": DragPage()},
    {"title": "Scale & Zoom", "page": ScalePage()},
    {"title": "Gesture Playground", "page": PlaygroundPage()},
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Gesture Lab")),
      body: Padding(
        padding: EdgeInsets.all(16),
        child: GridView.builder(
          itemCount: menus.length,
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2,
            crossAxisSpacing: 16,
            mainAxisSpacing: 16,
          ), // SliverGridDelegateWithFixedCrossAxisCount
          itemBuilder: (_, i) {
            return GestureDetector(
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (_) => menus[i]["page"] as Widget),
                );
              },
              child: AnimatedContainer(
                duration: Duration(milliseconds: 300),
                decoration: BoxDecoration(
                  borderRadius: BorderRadius.circular(20),
                  gradient: LinearGradient(
                    colors: [Colors.blue, Colors.purple],
                  ), // LinearGradient
                ), // BoxDecoration
                child: Center(
                  child: Text(
                    menus[i]["title"] as String,
                    style: TextStyle(color: Colors.white, fontSize: 16),
                    textAlign: TextAlign.center,
                  ), // Text
                ), // Center
              ), // AnimatedContainer
```

KESIMPULAN

Aplikasi Gesture Lab menunjukkan bagaimana Flutter bisa menangani berbagai interaksi pengguna secara langsung.

- TapPage: dasar interaksi tap, double tap, dan long press.
- DragPage: menggeser objek di layar.
- ScalePage: memperbesar, memperkecil, dan menggeser objek menggunakan pinch dan scroll.
- PlaygroundPage: menggabungkan semua gesture menjadi satu halaman interaktif.

HomePage menyatukan semua fitur dengan tampilan menu yang rapi dan animasi halus, sehingga pengguna bisa dengan mudah menjelajahi dan mencoba berbagai gesture Flutter secara menyenangkan.

TERIMA KASIH

SEMOGA BERMANFAAT